Operating System

Lab 8

Fasiha Tariq – 53289

Submitted to:

Ma’am Ayesha Ikram

Date: October 6th,2025

TASKS

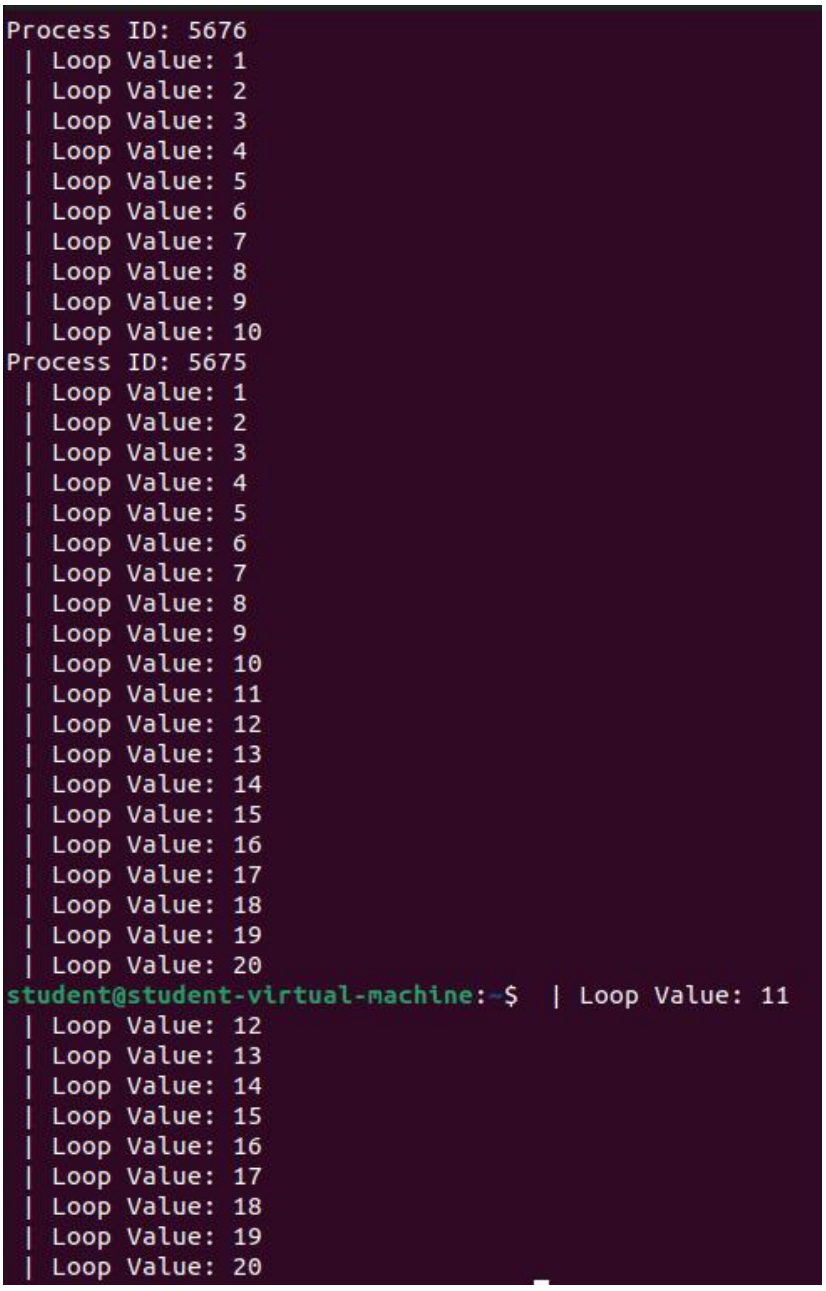
1. File name 1.c Write a C++ program that uses two fork() calls . Each process should print its process ID (PID) and a loop value from 1 to 20.

```
student@student-virtual-machine:~$ pico 1.cpp
student@student-virtual-machine:~$ cat 1.cpp
#include<iostream>
#include<unistd.h>
using namespace std;

int main()
{
fork();//First
fork ();//Second

pid_t pid = getpid();
cout << "Process ID: " << pid << endl;

for(int i=1; i<=20; i++)
{
cout<< " | Loop Value: " << i << endl;
}
return 0;
}
student@student-virtual-machine:~$ g++ 1.cpp -o Task1
student@student-virtual-machine:~$ ./Task1
Process ID: 5674
 | Loop Value: 1
 | Loop Value: 2
 | Loop Value: 3
 | Loop Value: 4
 | Loop Value: 5
 | Loop Value: 6
 | Loop Value: 7
 | Loop Value: 8
 | Loop Value: 9
 | Loop Value: 10
 | Loop Value: 11
 | Loop Value: 12
 | Loop Value: 13
 | Loop Value: 14
 | Loop Value: 15
 | Loop Value: 16
 | Loop Value: 17
 | Loop Value: 18
 | Loop Value: 19
 | Loop Value: 20
Process ID: 5677
 | Loop Value: 1
 | Loop Value: 2
 | Loop Value: 3
 | Loop Value: 4
 | Loop Value: 5
 | Loop Value: 6
 | Loop Value: 7
 | Loop Value: 8
 | Loop Value: 9
 | Loop Value: 10
 | Loop Value: 11
 | Loop Value: 12
 | Loop Value: 13
 | Loop Value: 14
 | Loop Value: 15
 | Loop Value: 16
 | Loop Value: 17
 | Loop Value: 18
 | Loop Value: 19
 | Loop Value: 20
```


Process ID: 5676
| Loop Value: 1
| Loop Value: 2
| Loop Value: 3
| Loop Value: 4
| Loop Value: 5
| Loop Value: 6
| Loop Value: 7
| Loop Value: 8
| Loop Value: 9
| Loop Value: 10
Process ID: 5675
| Loop Value: 1
| Loop Value: 2
| Loop Value: 3
| Loop Value: 4
| Loop Value: 5
| Loop Value: 6
| Loop Value: 7
| Loop Value: 8
| Loop Value: 9
| Loop Value: 10
| Loop Value: 11
| Loop Value: 12
| Loop Value: 13
| Loop Value: 14
| Loop Value: 15
| Loop Value: 16
| Loop Value: 17
| Loop Value: 18
| Loop Value: 19
| Loop Value: 20
student@student-virtual-machine:~$ | Loop Value: 11
| Loop Value: 12
| Loop Value: 13
| Loop Value: 14
| Loop Value: 15
| Loop Value: 16
| Loop Value: 17
| Loop Value: 18
| Loop Value: 19
| Loop Value: 20

2. File name 2.c. Write a C++ program that creates three child processes using the fork() system call. Each child's process should:

Print its own process ID (PID) and its parent process ID (PPID).

Terminate using exit().

After creating the child processes, the parent process should print its own PID.

```
student@student-virtual-machine:~$ pico 2.cpp
student@student-virtual-machine:~$ cat 2.cpp
#include <iostream>
#include <unistd.h>
#include <stdlib.h>
using namespace std;

int main()
{
    pid_t pid;

    for (int i = 1; i <= 3; i++) {
        pid = fork();
        if (pid == 0) {
            // Child process
            cout << "Child " << i << " PID: " << getpid()
                 << " | Parent PID: " << getppid() << endl;
            exit(0);
        }
    }

    // Parent process prints its own PID
    cout << "Parent process PID: " << getpid() << endl;
    return 0;
}
student@student-virtual-machine:~$ g++ 2.cpp -o Task2
student@student-virtual-machine:~$ ./Task2
Child 1 PID: 5811 | Parent PID: 5810
Parent process PID: 5810
Child 2 PID: 5812 | Parent PID: 1558
student@student-virtual-machine:~$ Child 3 PID: 5813 | Parent PID: 1558
```

3. Explain the working of system calls with their types and examples according to your understanding.

Answer:

System call is a method for a user program to ask the operating system kernel for services, like process creation, file reading, or device handling. User programs are not able to access the hardware directly, and therefore, system calls provide an interface between user mode and kernel mode.

Working:

When a system is invoked, the CPU mode is switched from user mode to kernel mode, the OS executes the desired action (such as opening a file or creating a process), and then execution returns to the program.

Types:

- Process Control: fork(), exec(), exit(), wait()
- File Management: open(), read(), write(), close()
- Device Management: ioctl(), read(), write()
- Process Management: getpid(), getuid(), getgid(), alarm()
- Communication: pipe(), msgsnd()
- Signal Handling: kill(), signal(), alarm()

Examples:

fork() → creates a new process

read() → reads data from a file

exit() → ends a process